CCP Estimation Primer

Hotz-Miller Inversion and Nested Pseudo-Likelihood

EconIRL Technical Notes

1 Problem and Motivation

The Nested Fixed Point (NFXP) estimator of Rust (1987) is statistically efficient but computationally expensive. At each outer optimization step, NFXP solves the Bellman equation to machine precision via an inner loop whose cost scales as $O(|\mathcal{S}|^2/(1-\beta))$. For large state spaces or high discount factors this inner loop dominates computation time, taking minutes per likelihood evaluation.

The CCP approach eliminates the inner loop entirely. Hotz and Miller (1993) showed that under additive separability and IID private shocks, the observed choice probabilities $P(a|s)$ contain all the information needed to recover continuation values without solving any fixed point. The idea is to invert the mapping from value functions to choice probabilities, rather than solve forward from parameters to values to probabilities. This inversion replaces thousands of Bellman iterations with a single matrix solve of cost $O(|\mathcal{S}|^3)$, making estimation dramatically faster for moderate state spaces.

Aguirregabiria and Mira (2002) showed that iterating the Hotz-Miller procedure in CCP space, a process they called Nested Pseudo-Likelihood (NPL), converges to the maximum likelihood estimator under regularity conditions. A single Hotz-Miller step ($K=1$) gives a consistent but inefficient estimator. Five to ten NPL iterations ($K=5-10$) recover full MLE efficiency.

CCP estimation is the only method in the EconIRL package that extends naturally to dynamic games, where NFXP requires computing Nash equilibria in the inner loop and is therefore infeasible.

2 Model and Notation

Table 1: Common DDC notation shared across all primers.

Symbol	Name	Definition
$s \in \mathcal{S}$	State	Discrete or continuous
$a \in \mathcal{A}$	Action	Finite: $\lvert\mathcal{A}\rvert = J$
s'	Next state	Drawn from $P(s' \mid s, a)$
$\beta \in [0, 1)$	Discount factor	
$\sigma > 0$	Scale parameter	Logit noise scale ($= 1$ by convention)
$\theta \in \Theta \subset \mathbb{R}^{d_\theta}$	Structural parameters	Unknown
$\phi(s, a)$	Feature vector	Known functions of (s, a), dimension K
$u(s, a; \theta)$	Flow utility	$\theta^\top \phi(s, a)$ (linear-in-parameters)
$P(s' \mid s, a)$	Transition kernel	Known or estimated from data
$Q(s, a)$	Choice-specific value	$u(s, a; \theta) + \beta \sum_{s'} P(s'\lvert s, a)\, V(s')$
$V(s)$	Integrated value	$\sigma \log \sum_a \exp(Q(s, a)/\sigma)$
$\pi(a \mid s)$	Choice probability	$\exp\big((Q(s, a) - V(s))/\sigma\big)$
τ	Trajectory	$(s_0, a_0, s_1, a_1, \ldots)$ from one agent
$\mathcal{D}$	Panel data	Collection of N observed trajectories

We work within the DDC framework of Rust (1987). An agent in state $s \in \mathcal{S}$ chooses action $a \in \mathcal{A}$ to maximize discounted expected utility. Flow utility is $u(s, a; \theta) + \sigma\epsilon_a$ where ϵ_a is Type I extreme value with scale $\sigma > 0$. Transitions follow $P(s'|s, a)$ with discount factor $\beta \in [0, 1)$.

The key objects are as follows.

Symbol	Name	Definition
$e(a, s)$	Emax correction	$\gamma_{\text{Euler}} - \log \pi(a\|s)$, where $\gamma_{\text{Euler}} \approx 0.5772$
F_π	Policy-weighted transitions	$F_\pi[s, s'] = \sum_a \pi(a\|s)\, P(s'\|s, a)$

3 Core Theory

3.1 The Hotz-Miller Inversion Lemma

Under additive separability (AS), conditional independence (CI), and IID extreme value errors (EV), the integrated value function at state s under policy π can be written as

$$\bar{v}(s) = \sum_a \pi(a|s)\big[u(s, a; \theta) + e(a, s)\big] + \beta \sum_{s'} F_\pi(s, s')\, \bar{v}(s'). \tag{1}$$

The emax correction $e(a, s) = \gamma_{\text{Euler}} - \log \pi(a|s)$ is entirely determined by the CCPs and the known distributional assumption on ϵ. No knowledge of θ or V is needed to compute it.

Rearranging Equation (1) in matrix form gives

$$\bar{v} = (I - \beta\, F_\pi)^{-1} \left[\sum_a \pi(a) \odot \big(u(a; \theta) + e(a)\big) \right], \tag{2}$$

where $\odot$ denotes element-wise multiplication and $\pi(a)$, $u(a; \theta)$, and $e(a)$ are $|\mathcal{S}|$-dimensional vectors for each action a. The matrix $(I - \beta F_\pi)$ is invertible because $\beta < 1$ and F_π is a stochastic matrix, so its spectral radius is at most 1.

This is the Hotz-Miller inversion: given CCPs $\pi(a|s)$, the continuation value $\bar{v}(s)$ is recovered by a single matrix inversion without solving the Bellman equation.

3.2 Choice-Specific Values from CCPs

For linear utility $u(s, a; \theta) = \theta^\top \phi(s, a)$, the choice-specific value function has the CCP representation

$$v(s, a) = \theta^\top \tilde{\phi}(s, a) + \tilde{e}(s, a), \tag{3}$$

where the "augmented" features and emax corrections are

$$\tilde{\phi}(s, a) = \phi(s, a) + \beta \sum_{s'} P(s'|s, a)\, W_\phi(s'), \tag{4}$$

$$\tilde{e}(s, a) = \beta \sum_{s'} P(s'|s, a)\, W_e(s'), \tag{5}$$

with $W_\phi = (I - \beta F_\pi)^{-1} \big[\sum_a \pi(a) \odot \phi(a)\big]$ of dimension $(|\mathcal{S}|, K)$ and $W_e = (I - \beta F_\pi)^{-1} \big[\sum_a \pi(a) \odot e(a)\big]$ of dimension $(|\mathcal{S}|,)$.

This decomposition separates the part of $v(s, a)$ that is linear in θ from the part that depends only on the CCPs and the known error distribution. The augmented features $\tilde{\phi}$ play the same role as the features ϕ in a static logit, but they incorporate the forward-looking continuation value via the matrix inversion.

3.3 NPL Convergence (Aguirregabiria-Mira 2002)

Aguirregabiria and Mira (2002) define the NPL mapping $\Psi : \pi \mapsto \pi'$ as follows. Given CCPs $\pi^{(k)}$, maximize the pseudo-log-likelihood

$$\hat{\theta}^{(k)} = \arg\max_{\theta} \sum_{i=1}^{N} \log \frac{\exp\big(v(s_i, a_i;\, \theta, \pi^{(k)})/\sigma\big)}{\sum_{a'} \exp\big(v(s_i, a';\, \theta, \pi^{(k)})/\sigma\big)}, \tag{6}$$

then update $\pi^{(k+1)}(a|s) = \mathrm{softmax}\big(v(s, a;\, \hat{\theta}^{(k)}, \pi^{(k)})/\sigma\big)$.

A fixed point of Ψ is an NPL equilibrium. Aguirregabiria and Mira (2002) prove convergence under Lyapunov stability: if the spectral radius of the Jacobian of Ψ at the fixed point is less than 1, NPL converges. The **zero Jacobian property** ensures that at the NPL fixed point, the estimator is first-order insensitive to CCP estimation error. This means the NPL fixed point achieves the same asymptotic distribution as NFXP-MLE, despite starting from noisy empirical CCPs.

3.4 Identification

CCP estimation inherits the same identification structure as NFXP. The parametric restriction $u = \theta^\top \phi(s, a)$ with $K < |\mathcal{S}|(|\mathcal{A}| - 1)$ resolves the reward shaping ambiguity. The transition matrix $P(s'|s, a)$ must be known or consistently estimated from data. The discount factor β and scale parameter σ are typically fixed rather than estimated, following Magnac and Thesmar (2002) who showed that β and the reward level are not jointly identified without further restrictions.

4 Estimation Algorithm

Algorithm 1: CCP-NPL Estimation (Hotz and Miller, 1993; Aguirregabiria and Mira, 2002)

Input: Panel $\mathcal{D} = \{(s_i, a_i)\}_{i=1}^N$, features $\phi(s,a)$, transitions $P(s'|s,a)$, discount β, scale σ, NPL iterations K

```
// Step 1: Estimate CCPs from data
```

for *each state* s **do**

  $\hat{\pi}^{(0)}(a|s) = N(s,a)/N(s)$ `// frequency estimator`

  if $N(s) < n_{\min}$ **then** $\hat{\pi}^{(0)}(a|s) = 1/|\mathcal{A}|$ `// fallback to uniform`

end

Apply Laplace smoothing: $\hat{\pi}^{(0)} \leftarrow (\hat{\pi}^{(0)} + \delta)/(1 + |\mathcal{A}|\delta)$

```
// Step 2: NPL loop
```

for $k = 0, 1, \ldots, K-1$ **do**

```
// 2a. Compute emax corrections from current CCPs
```

$e^{(k)}(a,s) = \gamma_{\text{Euler}} - \log \hat{\pi}^{(k)}(a|s)$

```
// 2b. Compute policy-weighted transitions
```

$F_\pi^{(k)}[s,s'] = \sum_a \hat{\pi}^{(k)}(a|s)\, P(s'|s,a)$

```
// 2c. Matrix inversion for continuation values
```

$W_\phi = (I - \beta F_\pi^{(k)})^{-1} \sum_a \hat{\pi}^{(k)}(a) \odot \phi(a)$ `//` $(|\mathcal{S}|, K)$

$W_e = (I - \beta F_\pi^{(k)})^{-1} \sum_a \hat{\pi}^{(k)}(a) \odot e^{(k)}(a)$ `//` $(|\mathcal{S}|,)$

```
// 2d. Compute augmented features and emax
```

for *each* (s,a) **do**

  $\tilde{\phi}(s,a) = \phi(s,a) + \beta \sum_{s'} P(s'|s,a)\, W_\phi(s')$

  $\tilde{e}(s,a) = \beta \sum_{s'} P(s'|s,a)\, W_e(s')$

end

```
// 2e. Maximize pseudo-log-likelihood over θ
```

$v(s,a;\theta) = \theta^\top \tilde{\phi}(s,a) + \tilde{e}(s,a)$

$\hat{\theta}^{(k)} = \arg\max_\theta \sum_{i=1}^N \log \operatorname{softmax}\big(v(s_i, \cdot;\, \theta)/\sigma\big)_{a_i}$ `// L-BFGS-B`

```
// 2f. Update CCPs for next iteration (NPL)
```

$\hat{\pi}^{(k+1)}(a|s) = \operatorname{softmax}\big(v(s,a;\, \hat{\theta}^{(k)})/\sigma\big)$

```
// 2g. Convergence check
```

if $\|\hat{\theta}^{(k)} - \hat{\theta}^{(k-1)}\| < \varepsilon$ **then break**

end

```
// Step 3: Compute value function and Hessian
```

$V(s) = \sigma \log \sum_a \exp\big(v(s,a;\, \hat{\theta})/\sigma\big)$

Compute numerical Hessian H at $\hat{\theta}$ via the full Bellman-constrained log-likelihood

Standard errors: $\mathrm{SE}(\hat{\theta}) = \sqrt{\operatorname{diag}((-H)^{-1})}$

Output: $\hat{\theta}$, V^*, $\hat{\pi}$, standard errors

Implementation notes. The EconIRL implementation uses float64 arithmetic when $\beta > 0.99$ because the condition number of $(I - \beta F_\pi)$ scales as $(1-\beta)^{-1}$, reaching 10^4 at $\beta = 0.9999$. The pseudo-log-likelihood gradient in Step 2e is computed via central finite differences rather than JAX autodiff because the valuation matrix computation involves Python loops that are not JAX-traceable. For the Hessian in Step 3, the implementation uses the full Bellman-constrained log-likelihood (re-solving value iteration at each perturbation) rather than the pseudo-likelihood with fixed CCPs, because the pseudo-likelihood Hessian is locally flat in directions that primarily affect CCPs, producing spurious rank deficiency. The `num_policy_iterations` parameter controls K: set $K=1$ for Hotz-Miller, $K=5-10$ for NPL, or $K=-1$ for convergence-based stopping.

5 Results and Comparison

Table 2: CCP-NPL ($K = 10$) and Hotz-Miller ($K = 1$) versus NFXP on Rust bus (90 states, $\beta = 0.9999$, 20,000 observations). True parameters: $\theta_c = 0.001$, $RC = 3.0$.

Metric	NFXP	CCP-NPL	Hotz-Miller
$\hat{\theta}_c$	0.001231	0.001231	-0.000830
$\hat{RC}$	3.0115	3.0115	2.8568
SE(θ_c)	0.000309	0.000307	—
SE(RC)	0.051597	0.051422	—
Log-likelihood	-4263.20	-4263.20	-4285.25
Cosine similarity	1.0000	1.0000	1.0000
Wall time (s)	2.2	34.2	9.4
Speedup vs NFXP	—	0.1×	—

CCP-NPL and NFXP converge to identical parameter estimates and log-likelihood, confirming that NPL reaches the MLE fixed point. The Hotz-Miller one-step estimator ($K=1$) is consistent but uses the noisy empirical CCPs directly, producing slightly different estimates. For rapid specification search, $K=1$ is sufficient. For publication-grade estimates, $K=5-10$ is recommended.

CCP is the method of choice when inner-loop cost is the bottleneck, which occurs at high discount factors ($\beta > 0.99$) or moderate state spaces (100 to 500 states). CCP requires the full transition matrix, which limits its applicability to settings where transitions are known or can be estimated from data. For continuous state spaces where transition densities are hard to estimate, TD-CCP is the appropriate alternative.

A known limitation is that CCP-NPL can produce unreliable counterfactual predictions at high discount factors. On the Rust bus with $\beta = 0.9999$, a Type 3 counterfactual (doubling the replacement cost) produces a welfare change of -6737 from CCP compared to -381 from NFXP. The amplification arises because small parameter errors compound through the matrix inversion $(I - \beta F_\pi)^{-1}$ when β is near 1. For counterfactual analysis at high discount factors, NFXP is more reliable.

6 EconIRL Implementation

The CCP estimator is implemented in `src/econirl/estimation/ccp.py` as the `CCPEstimator` class. The companion script `ccp_results.py` in this folder generates Table 2 and all numerical macros used in this document. To regenerate results, run `python docs/primers/ccp/ccp_results.py`.

Table 3: Mapping between algorithm steps and code.

Algorithm Step	Method	Key Operation
Step 1 (CCPs)	`_estimate_ccps_from_data()`	$N(s,a)/N(s)$ with smoothing
Step 2a (emax)	`_compute_emax_correction()`	$\gamma - \log \hat{\pi}$
Step 2b–c (inversion)	`_compute_valuation_matrix()`	$(I - \beta F_\pi)^{-1}$
Step 2d (augment)	`_compute_choice_specific_values()`	$\tilde{\phi} = \phi + \beta P W_\phi$
Step 2e (optimize)	`minimize_lbfgsb()`	L-BFGS-B
Step 2f (update)	`_update_ccps_from_values()`	softmax(v/σ)
Step 3 (Hessian)	`compute_numerical_hessian()`	Full Bellman LL

Usage.

```
from econirl.environments.rust_bus import RustBusEnvironment
from econirl.estimation.ccp import CCPEstimator
from econirl.preferences.linear import LinearUtility

env = RustBusEnvironment(
    operating_cost=0.001, replacement_cost=3.0,
    num_mileage_bins=90, discount_factor=0.9999,
)
panel = env.generate_panel(n_individuals=200, n_periods=100, seed=42)

ccp = CCPEstimator(num_policy_iterations=10, se_method="robust")
result = ccp.estimate(
    panel=panel,
    utility=LinearUtility.from_environment(env),
    problem=env.problem_spec,
    transitions=env.transition_matrices,
)
print(result.summary())
```

Setting `num_policy_iterations=1` gives the Hotz-Miller one-step estimator. Setting it to -1 iterates until convergence.

References

Aguirregabiria, V. and Mira, P. (2002). Swapping the Nested Fixed Point Algorithm: A Class of Estimators for Discrete Markov Decision Models. *Econometrica*, 70(4):1519–1543.

Aguirregabiria, V. and Mira, P. (2010). Dynamic Discrete Choice Structural Models: A Survey. *Journal of Econometrics*, 156(1):38–67.

Hotz, V. J. and Miller, R. A. (1993). Conditional Choice Probabilities and the Estimation of Dynamic Models. *The Review of Economic Studies*, 60(3):497–529.

Magnac, T. and Thesmar, D. (2002). Identifying Dynamic Discrete Decision Processes. *Econometrica*, 70(2):801–816.

Rust, J. (1987). Optimal Replacement of GMC Bus Engines: An Empirical Model of Harold Zurcher. *Econometrica*, 55(5):999–1033.